

Flask とは

Flask は Python で Web アプリケーションを作るためのフレームワークです。軽量で扱いやすくシンプルな構成のため柔軟に開発できるのが特徴です。

Flask では、HTML ファイルとの連携で Web ページの表示や、フォームの送受信処理、データベースとの連携、ログイン機能（Flask-Login などの拡張）が可能です。

Flask は MVC（Model-View-Controller）の設計・開発モデルの構造が一般的で「View → templates」「Controller → app.py」が対応します。

Flask の基本構成

■ 環境構築

1. 開発ディレクトリの作成

[ターミナル]

```
> mkdir flask_app
> cd flask_app
```

2. Flask ライブラリのインストール

[ターミナル]

```
flask_app> pip install flask
```

3. MySQL の接続ライブラリのインストール

[ターミナル]

```
flask_app> pip install mysqlclient
```

■ ディレクトリ構成

flask_app

- └ app.py
- └ config.py
- └ templates /
 - └ index.html
- └ static /
 - └ css /style.css
 - └ js /

app.py

アプリケーションのエントリーポイント。

Flask インスタンスを作成し、ルーティングやビューの関数を定義する。

config.py

アプリケーションの設定を管理。

データベースの接続管理、デバッグモード、セキュリティキーなどを記述する。

templates

HTML テンプレートを格納する。

Flask の `render_template()` で呼び出される。

index.html

トップページの HTML ファイル

python の変数やループ処理などを使って動的な表示が行える。

static

静的ファイルなどを格納する。

参照する場合 `<link>` タグではなく、 `url_for('static',filename='css/style.css')` で参照する。

css / style.css

web ページのスタイルを定義する。

js /

JavaScript ファイルを格納する。

Flask の基本コード

Flask では Controller である app.py を通じて各 HTML ファイルを呼び出します。
まずは、app.py を実行してブラウザで表示するまでの基本構成を理解しましょう。

■ app.py の作成

Flask-app の直下に app.py ファイルを作成します。

app.py

```
from flask import Flask    # Flask クラスのインポート

app = Flask(__name__)    # Flask インスタンスの生成

@app.route('/')    # ルーティングの定義
def hello():    # 関数の定義
    return "Hello Flask_app"

if __name__ == '__main__':    # このファイルが実行された場合だけアプリを起動する
    app.run(debug = True)    # エラーの詳細を表示する
```

■ 実行と Web ページの表示

ターミナルから app.py を実行します。

ターミナル

```
flask_app>python app.py
```

ブラウザ:URL

```
http://localhost:5000
```

ブラウザ表示

```
Hello Flask-app
```

Flask で HTML ファイル(Web ページ)を表示

■ index.html ファイルの表示

templates ディレクトリに index.html ファイルを作成します。

[templates/index.html](#)

```
<!DOCTYPE html>
<html lang="ja">
<head>
    <meta charset="UTF-8">
    <title>Flask-app</title>
</head>
<body>
    <h1>Flask-app</h1>
    <p>ここは、index.html です</p>
</body>
</html>
```

app.py に render_template モジュールをインポートし、hello()関数を index()関数に書き換えます。

[app.py](#)

```
from flask import Flask
from flask import render_template    # HTML テンプレートを表示するモジュール

@app.route('/')
def index():
    return render_template("index.html")    # templates フォルダの index.html を探す

# hello()関数を削除またはコメントアウトする
# def hello():
#     return "Hello Flask_app"

if __name__ == '__main__':
    app.run(debug = True)
```

実行して結果を表示します。

ターミナル

```
flask_app>python app.py
```

ブラウザ:URL

```
http://localhost:5000
```

ブラウザ表示

Flask-app

ここは、index.html です

Web ページの遷移（Get 送信）

基本コードでも解説しましたが、Flask では Controller である app.py を通じて各 HTML ファイルを呼び出します。そのためページを遷移する場合は app.py を通って次のページへ移動します。

ここでは、index ページから form ページへ遷移します。

■ form.html の作成

templates ディレクトリに form.html ファイルを作成します。

[templates/form.html](#)

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>Flask-app</title>
</head>
<body>
  <h1>Flask-app</h1>
  <p>ここは、form.html です</p>
</body>
</html>
```

■ index.html の追記

index.html ファイルに form.html へ遷移するボタンを作成します。

[templates/index.html](#) ※body タグより上部の記述は省略

```
<body>
  <h1>Flask-app</h1>
  <p>ここは、form.html です</p>
  <p><a href="/form">form ページへ</a></p>    <!-- 追記 -->
</body>
</html>
```

■ app.py の追記

app.py に form ページに遷移するための form()関数を作成します。

app.py

```
@app.route('/form', methods=['GET'])
def form():
    return render_template("form.html")
```

■ 遷移の確認

index ページのブラウザを更新して表示を確認しましょう。

ブラウザ表示

Flask-app

ここは、index.html です

[form ページへ](#)

[form ページへ](#)をクリックし、遷移するかを確認しましょう。

ブラウザ表示

Flask-app

ここは、form.html です

パラメータの受け渡し

ここでは、app.py でパラメータを設定し index.html で受け取って表示させる処理を作成します。

■ app.py でパラメータの設定

app.py の index()関数を以下のように書き換えます。

app.py

```
@app.route('/')
def index():
    params = {
        "title": "Flask-app",
        "msg": "これは、app.py で設定されたメッセージです。",
    }
    return render_template("index.html", **params)
```

辞書型変数 params を定義し、title キーに”Flask-app”、msg キーに”ここは、index ページです。”と値を設定して render_template() の第二引数に **params として可変長に設定します。

■ index.html の更新

送られたパラメータは index.html で辞書型変数のキーを {{ }} の中に指定して受け取ります。

templates/index.html

```
<!DOCTYPE html>
<html lang="ja">
<head>
    <meta charset="UTF-8">
    <title>{{ title }}</title>
</head>
<body>
    <h1>{{ title }}</h1>
    <p>ここは、index ページです。</p>
    <p>{{ msg }}</p>
    <p><a href="/form">form ページへ</a></p>
</body>
</html>
```


ブラウザ表示

Flask-app

ここは、index.html です

これは、app.py で設定されたメッセージです。

[form ページへ](#)

リクエストパラメータの取得と表示（直接レンダリングする場合）

form.html ファイルに入力フォームを作成、入力フォームから Post 送信された値を app.py で受け取り、result.html で表示する処理を作成しましょう。

■ form.html の追記

form.html ファイルに入力フォームを作成します。

[templates/form.html](#) ※body タグ内だけ記述

```
<body>
  <h1>Flask-app</h1>
  <p>ここは、form.html です</p>
  <form action="/form" method="post">
    <div>
      <input type="text" name="text">
      <input type="submit" value="送信">
    </div>
  </form>
</body>
```

■ app.py でリクエストパラメータの取得

app.py の form()関数を書き換えます。

[app.py](#)

```
@app.route('/form', methods=['GET','POST'])
def form():
    if request.method == 'GET':    # GET 送信されてきた場合
        return render_template("form.html")

    elif request.method == 'POST': # POST 送信されてきた場合
        text = request.form["text"]
        params = {
            "title":"Flask-app",
            "text": text,
        }
        return render_template("result.html",**params)
```

■ result1.html の作成

templates に result.html ファイルを作成します。

result.html

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{{ title }}</title>
</head>
<body>
  <h1>{{ title }}</h1>
  <p>ここは、result ページです。</p>
  <p>送られて来たパラメータは「{{ text }}」です。</p>
  <p><a href="/form">form ページへ</a></p> <!-- 追記 -->
</body>
</html>
```

ブラウザを更新して form.html へ遷移します。入力フォームに「テスト」と入力して送信しましょう。

ブラウザ表示：form.html

Flask-app

ここは、form.html です

ブラウザ表示：result.html

Flask-app

ここは、result ページです。

送られて来たパラメータは「テスト」です。

[form ページへ](#)

直接レンダリングした場合、URL は「http://localhost:5000/form」となります。

リクエストパラメータの取得と表示（リダイレクトする場合）

リダイレクト（redirect）は、ある URL へのアクセスを、別の URL へ自動的に転送する仕組みのことです。Web アプリやブラウザでよく使われます。

ここでは、form.html から POST 送信されたリクエストパラメータを result.html へリダイレクトさせます。

app.py の result()関数を作成します。

app.py

```
@app.route('/result', methods=['POST'])
def result():
    text = request.form["text"]
    params = {
        "title": "Flask-app",
        "text": text,
    }
    return render_template("result.html", **params)
```

form.html の form タグ内の href = “/result” に変更します。これで form.html から POST 送信されたり

クエストパラメータは app.py の result()関数で取得することになります。

[form.html](#) ※form タグ内だけ記述

```
<form action="/result" method="post">
  <div>
    <input type="text" name="text">
    <input type="submit" value="送信">
  </div>
</form>
```

表示される内容は同じですが、リダイレクトされた場合、URL は「http://localhost:5000/result」となります。

■ 直接レンダリングとリダイレクト

➤ 直接レンダリング

特徴

- ・ POST リクエストに対して、サーバー側でそのままテンプレートを描画して返す。
- ・ URL は /form のまま変わらない。

メリット

- ・ 実装がシンプル。
- ・ フォーム送信後すぐに結果を表示できる。

デメリット

- ・ ブラウザのリロード問題：ユーザーが再読み込みすると、再度 POST が送信されてしまう。
- ・ ユーザーが結果ページの URL を共有しても、POST データがないと再表示できない。

➤ リダイレクト

メリット

- ・ リロードしても再送信されない (POST → GET に切り替わる)。
- ・ URL が /result に変わり、結果ページとして明確になる。
- ・ ユーザーが結果ページをブックマーク・共有しやすい。

デメリット

- ・ 少しコードが複雑になる (session や query stringなどでデータを渡す必要あり)。

結論としてどちらを使うのが良いかは、使用する条件によって変わります。

- ・実装をシンプルに保ちたい
- ・一時的なデータの表示だけで良い場合は、直接レンダリングを使います。

- ・フォーム送信後の再読み込みを防ぎたい
- ・結果ページを URL で共有したい場合は、リダイレクトを使います。